

Doubly Robust Estimators Under Small Overlap

BGN: 6582P

1 Introduction

Doubly Robust (DR) estimators protect against potential model mis-specification when conducting causal inference in a discrete treatment setting. Parametric methods following Rubin’s potential outcome framework will typically model either the outcome or the treatment propensity separately. If mis-specified, the models are biased and causal effects are unidentified. On the other hand, DR estimators layer both an outcome and treatment propensity model to correct for potential biases. As long as at least one of the two models has been correctly specified, consistent estimates for the treatment effect can be recovered.

DR estimators still depend on key assumptions for causal identification. The stable unit treatment value assumption (SUTVA) says that potential outcomes depend only on the subject’s own treatment status, and not that of others (i.e. no spillovers). Unconfoundedness requires that, once we condition on nuisance parameters, the treatment outcome is independent of actual treatment assignment. This requires controlling for all endogeneity in treatment assignment. Finally, the overlap assumption says that there must be comparable individuals in both the treated and control groups who share similar characteristics. This identifies the counterfactual by comparing similar individuals. If all three assumptions are satisfied, then the causal effects can be identified by a correctly specified model.

However, the overlap assumption does not prescribe a minimum proportion of similar individuals that are needed to identify the causal effect. The purpose of this report is to investigate the finite sample performance of the DR estimator as overlap varies. I run a set of Monte Carlo simulations with a correctly specified model to examine the conditional average treatment effect (CATE) and average treatment effect (ATE) as overlap worsens, either directly through the data generating process or through variation in treatment prevalence.

I find that the bias caused by poor overlap is amplified when treatment effects are heteroge-

neous, and that this effect increases with the strength of heterogeneity. Additionally, I find that doubly robust learners suffer greatly in small samples when overlap is poor. This is especially true when the treatment is rare, and propensity scores are skewed. On the other hand, the DR estimator is shown to perform well when there is clear overlap, even in very small samples when treatment is strongly heterogeneous.

This suggests that the DR estimator is a strong candidate for causal inference when overlap is strong and sample size is small, as it offers us protection against model misspecification. However, as overlap deteriorates the errors from adding heterogeneity are amplified in finite samples. This makes a strong case for parametric methods when overlap is less certain, as parametric models do not amplify this bias.

The remainder of this report is structured as follows: chapter 2 provides a discussion of the DR estimator’s finite sample performance, chapter 3 describes the data generating process and methodology, chapter 4 presents the results from the Monte Carlo simulations, and chapter 5 is a discussion of the findings.

2 Background

2.1 Literature Review

The topic of poor overlap has been extensively explored for parametric methods. Li et al. (2019) and Zhou et al. (2020) explore alternative remedies for inverse probability weight (IPW) models. They suggest trimming tails of the propensity distribution, overlap weighting, entropy weighting, or switching to a matching model based on propensity scores. Both studies find overlap weights to be the most effective remedy.

Yang et al. (2026) run Monte Carlo simulations using the DR estimator for a homogeneous treatment, and compare it against the individual outcome and propensity models. They find that the IPW model is particularly sensitive to poor overlap and performs worse than the DR estimator. However, the outcome model outperforms the DR estimator when overlap is particularly bad.

To my knowledge, no study has investigated the role of overlap for the DR estimator when treatment effects are heterogeneous.

2.2 Finite Sample Performance of the DR Estimator

The DR estimator provides protection against model mis-specification by including both an outcome model and a propensity model for the ATE. This allows each model to correct the potential bias from mis-specification from the other, leading to consistent estimates as long as one of the models has been correctly specified. Additionally, as long as both models converge at a rate of at least $n^{-1/4}$, then the DR estimator is convergent at a rate of $n^{-1/2}$. This makes the DR estimator an appealing strategy when the true outcome and propensity models are unknown.

The doubly-robust estimator in this report uses augmented inverse probability weighting (AIPW) (Bang and Robins, 2005). The estimate for the ATE is given by:

$$\hat{\tau}_{DR} = \frac{1}{n} \sum_{i=1}^n \left\{ \hat{m}_1(X_i) + \frac{D_i(Y_i - \hat{m}_1(X_i))}{\hat{e}(X_i)} \right\} - \frac{1}{n} \sum_{i=1}^n \left\{ \hat{m}_0(X_i) + \frac{(1 - D_i)(Y_i - \hat{m}_0(X_i))}{1 - \hat{e}(X_i)} \right\}.$$

Where D_i denotes the binary treatment status, Y_i denotes the individual outcomes, X_i are the set of individual characteristics, and n is the total number of observations. For each term in the expression, the first component is from the outcome model, and the second is from the propensity model on the residuals. $\hat{m}_d(X_i)$ is the fitted outcome for binary treatment d under the outcome model. $\hat{e}(X_i)$ denotes the propensity score, which is usually unobserved.

$$\hat{e}(X_i) = P(D_i = 1 \mid X_i).$$

Yang et al. (2026) decompose the finite sample model error into the residuals originating from the propensity and outcome models separately. Let $\tau_S = \frac{1}{n} \sum_{i=1}^n (Y_i(1) - Y_i(0))$ denote the true sample ATE. Then, the model error can be defined as $\hat{\tau}_{DR} - \tau_S$, which can be written as:

$$\hat{\tau}_{DR} - \tau_S = \frac{1}{n} \sum_{i=1}^n \frac{D_i - \hat{e}(X_i)}{\hat{e}(X_i)} (Y_i(1) - \hat{m}_1(X_i)) + \frac{1}{n} \sum_{i=1}^n \frac{D_i - \hat{e}(X_i)}{1 - \hat{e}(X_i)} (Y_i(0) - \hat{m}_0(X_i)).$$

We can simplify this into two key components by defining the propensity and outcome residuals as:

$$R_i^e = D_i - \hat{e}(X_i), \quad R_i^y = \frac{Y_i(1) - \hat{m}_1(X_i)}{\hat{e}(X_i)} + \frac{Y_i(0) - \hat{m}_0(X_i)}{1 - \hat{e}(X_i)}.$$

Then the DR model error can be re-written as:

$$\hat{\tau}_{DR} - \tau_S = \frac{1}{n} \sum_{i=1}^n R_i^e R_i^y.$$

This representation provides an insight into how overlap will affect the DR model estimates.

R_i^e denotes the propensity model residual, and represents the difference between the estimated propensity score and whether an individual is actually treated. It is large when individuals have a propensity score that is opposite to their actual treatment status.

R_i^y denotes the outcome model residual, and it is the sum of weighted prediction errors in the outcome model, with the weights given by the inverse of the estimated propensity scores. Therefore, theory suggests that the DR estimator makes greater errors when propensity scores are near zero or one. Areas with poor overlap lead to higher model errors.

Additionally, the outcome residual is increasing in the actual outcome model errors. When treatment is heterogeneous, the actual treatment effect can be harder to predict, leading to an increase in the numerator. Therefore, we should also expect greater heterogeneity to lead to greater DR model errors.

Finally, the individual model errors are amplified by one another due to multiplication of the outcome residual and the propensity residual. We should expect to see that the errors arising from heterogeneity are amplified by the model, leading to worse finite sample estimates as the degree of heterogeneity increases and the outcome model makes larger errors. Even when the DR model has been correctly specified, this error amplification could lead to inaccurate estimates when heterogeneity is present and overlap is poor.

The DR estimator is not only inheriting the model errors from both the outcome and propensity models, it is also amplifying them. Hence, errors arising from poor overlap and heterogeneity will in general be larger in finite samples.

3 Methodology

3.1 Data Generating Process

I run a set of Monte Carlo simulations using a correctly specified DR estimator. I generate data in-line with Li et al. (2019) and Yang et al. (2026) to ensure that results are comparable. The key departure is the inclusion of heterogeneity in the strength of treatment. Rather than a binary treatment effect, I allow the treatment effect to vary based on individual features.

In particular, I draw six latent variables from a multivariate Gaussian distribution with zero mean, unit marginal variance, and pairwise correlations of 0.5. The latter three variables are then transformed into binary features taking zero values if below zero, and 1 otherwise.

Treatment assignment is determined by an independent Bernoulli process such that the probability of treatment satisfies the following logistic propensity model:

$$\text{logit}(e(X)) = \alpha_0 + d(0.2X_1 + 0.3X_2 + 0.4X_3 - 0.25X_4 - 0.3X_5 - 0.3X_6).$$

d is a parameter controlling the degree of overlap in the sample. Higher values of d indicate worse overlap as propensity scores are pushed closer to the extremes at 0 and 1. I also consider two different treatment prevalence rates, which also have an impact on overlap. Lower prevalence (rare treatment) worsens overlap as propensity scores are skewed toward 0. I achieve the desired treatment propensity by shifting the intercept term α_0 in the propensity model. This creates four overlap scenarios illustrated in figure 1.

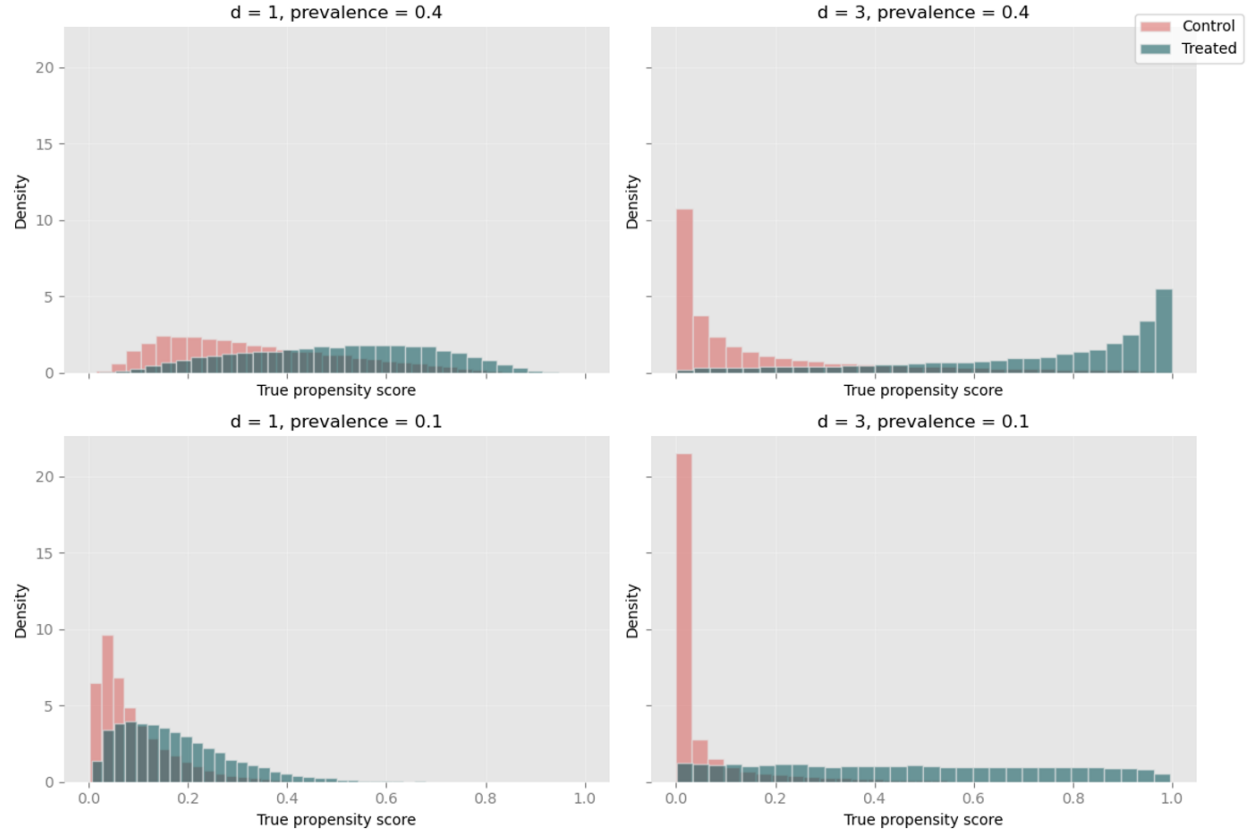
The top panels show scenarios when treatment prevalence is 40%. The right side panels show when overlap is low. Accordingly, the top-left panel shows the best overlap scenario and the lower-right panel shows the worst overlap. That is, when treatment is rare and propensity scores are skewed toward zero for the control group.

The untreated potential outcomes are given by a linear function of the following form:

$$Y_i(0) = -0.5X_{i1} - 0.8X_{i2} - 1.2X_{i3} + 0.8X_{i4} + 0.8X_{i5} + 1.0X_{i6} + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, 1).$$

Finally the true treatment effect is modelled as heterogeneous, depending only on X_1 for simplicity. This is the key departure from the existing literature. I vary the parameter h to

FIGURE 1: Simulated Overlap



prevalence corresponds to the proportion of the sample that is treated. Fewer treated leads to worse overlap. 'd' refers to the parameter in the logistic function controlling overlap. Higher values indicate worse overlap.

simulate varying strength of heterogeneity.

$$\tau(X_i) = 1 + hX_{i1}.$$

The full observed outcome equation is then given by:

$$Y_i = Y_i(0) + D_i \tau(X_i).$$

3.2 Model

The model is a DR learner trained using the *econML* package in Python. It is correctly specified given the data generating process, so that the propensity model is logistic and the outcome model is linear. The model is trained using 3-fold cross validation and the *lbfgs* solver.

Given a heterogeneous treatment, the estimated model outcomes are CATEs. At each replication, the model predicts the CATE for every observation. The RMSE is computed as:

$$\text{RMSE}_{CATE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{\tau}(X_i) - \tau(X_i))^2}.$$

I take the average RMSE across all replications and call this metric (CATE RMSE). This captures the model’s error in estimating heterogeneous treatment effects.

The estimated sample ATE can be computed by applying the Law of Iterated Expectations, and taking the mean over all CATEs for each replication. I compute the bias as the difference between the estimated and actual sample ATE τ_S . I take the mean over all replications and call this metric (ATE bias). This captures the size and direction of bias for the model’s average predictions.

Finally, I use ATE bias to compute the RMSE for ATE as:

$$\hat{\tau}_{ATE} = \frac{1}{n} \sum_{i=1}^n \hat{\tau}(X_i).$$

$$\text{RMSE}_{ATE} = \sqrt{\frac{1}{R} \sum_{r=1}^R \left(\hat{\tau}_{ATE}^{(r)} - \tau_S^{(r)} \right)^2}.$$

Where r denotes the current replication. I call this metric the ATE RMSE.

I replicate each simulation 1000 times and report the main results using a sample size of 1000.

4 Results

Table 1a shows results when treatment prevalence is 40%. The first row shows a baseline model performance when no heterogeneity is present. These results are negligibly different from those presented in Yang et al. (2026) which reassures us that the data generating process and methodology are consistent with this paper.

As heterogeneity increases, we observe that the model errors also increase, all else being equal. Focusing on the left-hand side columns (when overlap is at its best) we see that the errors from increased heterogeneity remain small as we move down the table.

TABLE 1A: Monte Carlo Simulations for 40% treatment prevalence

Heterogeneity Coefficient	High overlap (d=1)			Poor overlap (d=3)		
	ATE RMSE	ATE bias	CATE RMSE	ATE RMSE	ATE bias	CATE RMSE
0	0.076885	-0.000533	0.200646	0.396016	-0.006599	0.585291
0.5	0.080147	-0.001725	0.209276	0.545089	-0.033129	0.703677
1	0.085984	-0.009473	0.230153	0.723276	-0.089845	0.963303
2	0.105725	-0.018216	0.287694	1.157280	-0.111568	1.422709

Results shown for sample of 1000. All else equal, greater heterogeneity in treatment effects results in greater RMSE. Error from heterogeneity is amplified when overlap is poor.

TABLE 1B: Monte Carlo Simulations for 10% treatment prevalence

Heterogeneity Coefficient	High overlap (d=1)			Poor overlap (d=3)		
	ATE RMSE	ATE bias	CATE RMSE	ATE RMSE	ATE bias	CATE RMSE
0	0.192684	0.000965	0.477177	43.034530	1.695425	10.722467
0.5	0.221392	-0.051397	0.550236	54.653998	-2.856936	15.561054
1	0.360478	-0.129710	0.821067	77.385386	-2.529381	20.450878
2	0.584636	-0.253731	1.383130	198.672785	-15.736825	43.754794

Results shown for sample of 1000. When treatment prevalence is low and overlap is poor, we essentially face a full violation of the overlap assumption. The model is completely unable to identify the CATEs or ATE.

The right-hand side columns show the same models estimated when overlap is poor. Firstly, the top row shows that, when there is no heterogeneity, poor overlap alone causes higher model errors. Secondly, we observe that the errors arising from heterogeneity are amplified compared to the case with good overlap. Thirdly, when heterogeneity is at its strongest, the CATE RMSE increases by almost 50% relative to the next highest heterogeneity level. This suggests that greater heterogeneity paired with poor overlap can lead to substantial model errors, even when treatment prevalence is relatively balanced.

Table 1b shows the results when treatment prevalence is rare, leading to worse overlap. When overlap is poor, the model errors arising from greater heterogeneity are amplified. Greater heterogeneity leads to higher RMSE for both the CATE and ATE, which agrees with our theoretical predictions.

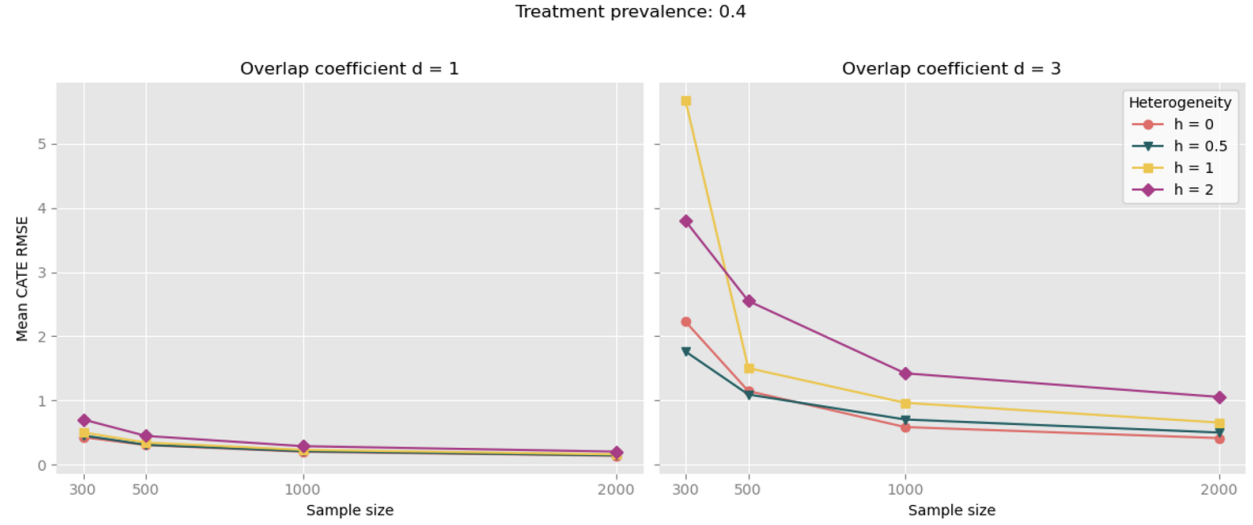
The right-hand columns show the worst overlap scenario, where control units all have propensities near zero, and only the treated have higher propensities. In this case, the model is shown to be practically useless, even when no heterogeneity is present. Overlap is so poor, and the propensities are so skewed that the model fails to identify the ATE or CATE at all. The model's errors explode as we add increasing degrees of heterogeneity to the true treatment effects.

In this case, the propensity model errors are large, and are amplifying the outcome model

errors from heterogeneity. This results in very poor model performance.

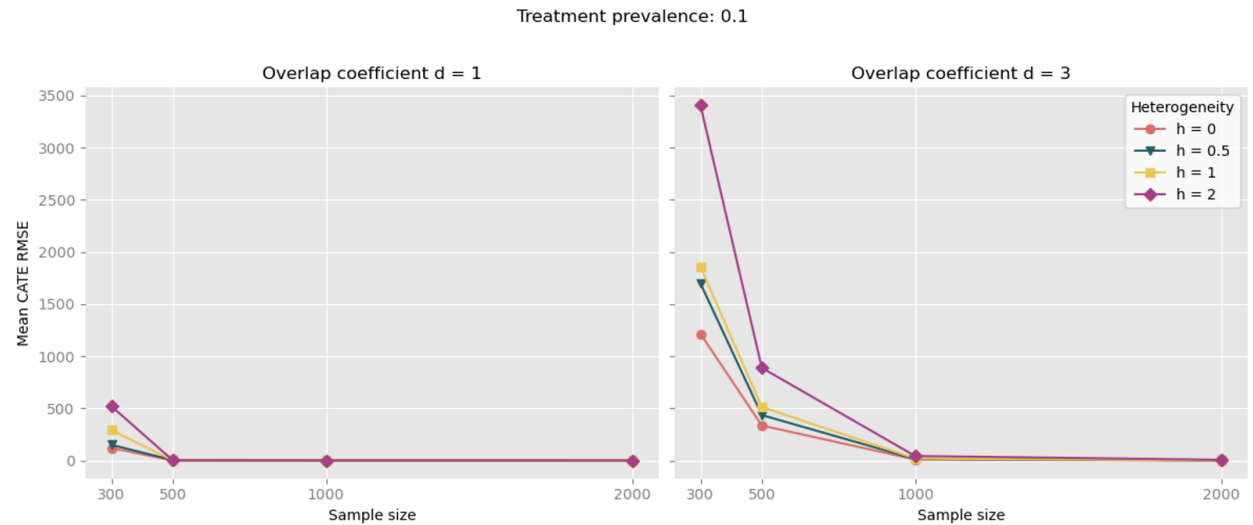
Figures 2 and 3 show how model performance varies as sample size varies.

FIGURE 2: Doubly Robust RMSE as sample size grows (high treatment prevalence)



The heterogeneity parameter 'h' corresponds to the coefficient on X_1 in the treatment effect. Higher values indicate stronger heterogeneous effects. In general, greater heterogeneity leads to worse estimates. This is compounded when overlap is poor.

FIGURE 3: Doubly Robust RMSE as sample size grows (low treatment prevalence)



The heterogeneity parameter 'h' corresponds to the coefficient on X_1 in the treatment effect. Higher values indicate stronger heterogeneous effects. When treatment prevalence is low, doubly robust estimators suffer in small samples.

Figure 2 shows that, in general, greater heterogeneity leads to greater model errors. These errors are particularly pronounced in small samples, as there is not enough variation in the sample to identify causal effects. If we suspect strong heterogeneity in treatment effects, obtaining larger samples is of particular importance.

On the other hand, when overlap is very good the DR estimator is shown to achieve a low

RMSE in small samples even when heterogeneity is present. This suggests that heterogeneity is not an issue when overlap between subjects is clearly satisfied. We may be content with a smaller sample if we can show clear overlap.

Figure 3 shows the case when treatment prevalence is rare. Note that this is on a different scale due to the fact that model RMSE explodes in very small sample sizes. In small samples, a 10% prevalence makes up a very small number of observations. This makes it difficult to find similar individuals for identifying the counterfactual, and this is worsened when heterogeneity is present.

5 Discussion

Overall, the simulation results agree with the theoretical hypothesis that errors arising from heterogeneity are amplified by the DR estimator. In particular, stronger heterogeneity leads to greater model errors, which are amplified when overlap is poor. This motivates additional caution when estimating heterogeneous treatment effects with the DR estimator when satisfaction of the overlap assumption is uncertain.

Additionally, I find that the errors from heterogeneity are greater in smaller samples. This agrees with the intuition that a smaller population provides less variation to identify individual treatment effects. However, I found that this is not such an issue when the overlap assumption is comfortably satisfied. In the best overlap scenario, I showed that the CATE RMSE remains low even when there are strongly heterogeneous effects. This highlights that the only case when we may be satisfied with a small sample is when SUTVA and unconfoundedness are satisfied, and there is substantial overlap between groups. As overlap worsens, so does finite-sample performance.

Finally, I found that the overlap scenario where propensities are concentrated near zero is particularly fatal to the DR estimator. Regardless of the degree of heterogeneity, the DR model was unable to identify the causal effect when treatment prevalence was low and propensities were skewed. This again agrees with the hypothesis that the DR model amplifies errors from the outcome and propensity models, and that these errors are most pronounced when propensities take on extreme values.

In light of these findings, it is clear that one should take caution with DR estimators when it is suspected that overlap is poor and heterogeneity is present. The DR estimator on its own is not equipped to deal with this issue. Yang et al. (2026) suggest that the best way to deal with this issue is to shift the target population onto one with better overlap. One

method of doing so is by trimming the tails of the propensity distribution to remove extreme propensity scores. This is shown to reduce RMSE, but it is undesirable in small samples as it entails removing a portion of the data.

Zhou et al. (2020) discuss overlap weights as a potential remedy to poor overlap for the inverse probability weights estimator. This entails down-weighting observations in the tails and up-weighting observations with high overlap. This reduces model error from poor overlap without removing observations.

Further research could investigate how effective these methods are when applied to the DR estimator with heterogeneous treatment effects, and how this varies for samples of varying sizes.

In conclusion, I have shown that heterogeneous effects cause greater model errors for the DR estimator in finite samples, and that this error is amplified when overlap is poor. The errors from heterogeneity and poor overlap are particularly pronounced in small samples, and when treatment is rare. This error amplification arises due to the combination of an output and propensity model; hence, parametric methods should be strongly considered in such applications. On the other hand, when overlap is comfortably satisfied the DR estimator is shown to perform well even with heterogeneous treatment effects in small samples, and has the additional benefit of protecting against model misspecification.

References

- [1] Bang, Heejung, and James M. Robins. Doubly robust estimation in missing data and causal inference models. *Biometrics*, 61(4):962–973, 2005.
- [2] Li, Fan, Laine E. Thomas, and Fan Li. Addressing extreme propensity scores via the overlap weights. *American Journal of Epidemiology*, 188(1):250–257, 2019.
- [3] OpenAI. ChatGPT. *Large language model*, 2026. Available at: <https://chat.openai.com/>. Accessed March 27, 2026.
- [4] Yang, Chengxin, Laine E. Thomas, and Fan Li. Demystify doubly-robust estimation: The role of overlap. *arXiv preprint arXiv:2602.01648*, 2026.
- [5] Zhou, Yunji, Roland A. Matsouaka, and Laine E. Thomas. Propensity score weighting under limited overlap and model misspecification. *Statistical Methods in Medical Research*, 29(12):3721–3756, 2020. PMID: 32693715.

A Code Appendix

A.1 orchestrator.py

The main orchestrator script for running the Monte Carlo simulations.

```
1 from pathlib import Path
2 from itertools import product
3 import pandas as pd
4
5 from overlapsim.data import SimulationData
6 from overlapsim.simulation import run_simulation
7 from overlapsim.metrics import summarise_results
8 from overlapsim.plot_utils import apply_gray_style
9
10
11 # Project root and results directory
12 project_root = Path.cwd()
13 results_dir = project_root / "results"
14
15
16 #####
17 # EXAMPLE: Running a single simulation with 1000 replications
18 #####
19
20 # Initialise the simulation data class
21 gen = SimulationData(
22     n_samples=500,
23     overlap_coef=1,
24     treatment_prevalence=0.1,
25     heterogeneity_coef=0.5
26 )
27
28 # Replicate the simulation 1000 times.
29 results = run_simulation(gen, n_replications=1000, base_seed=0)
30
31 # Create a summary of performance metrics for the full simulation run
32 summary = summarise_results(results)
33
34
```

```

35 #####
36 # Running all simulations
37 #####
38
39 heterogeneity_vals = [0, 0.5, 1, 2] # degree of heterogeneity
40 overlap_vals = [1, 3] # degree of overlap
41 treatment_prev_vals = [0.1, 0.4] # treatment prevalence
42 n_sample_vals = [300, 500, 1000, 2000] # number of samples
43
44 all_results = []
45
46 # Run a simulation once for every combination in grid
47 for i, (heterogeneity_coef, overlap_coef, treatment_prevalence,
48     n_samples) in enumerate(
49     product(heterogeneity_vals, overlap_vals, treatment_prev_vals,
50         n_sample_vals)
51 ):
52     # Display the simulation being run to keep track of where the
53     # loop is at
54     print(
55         f"[{i+1}/64] "
56         f"heterogeneity={heterogeneity_coef}, "
57         f"overlap={overlap_coef}, "
58         f"treatment_prevalence={treatment_prevalence}, "
59         f"n_samples={n_samples}"
60     )
61
62 # Initialise the data generating process with selected parameters
63 gen = SimulationData(
64     n_samples=n_samples,
65     overlap_coef=overlap_coef,
66     treatment_prevalence=treatment_prevalence,
67     heterogeneity_coef=heterogeneity_coef,
68 )
69
70 # Replicate 1000 times per simulation
71 results = run_simulation(
72     generator=gen,

```

```

70         n_replications=1000,
71         base_seed=123 + 10000 * i, # vary the base seed over
           simulations
72     )
73
74     # Append results over each loop to a single list
75     all_results.append(results)
76
77 # Concatenate the results to a master dataset
78 full_results = pd.concat(all_results, ignore_index=True)
79
80 # Save the master dataset
81 full_results.to_csv(results_dir /
           "simulation_replication_results.csv", index=False)
82
83
84 # Apply the summary function to the dataset for each parameter
           combination
85 # Will create one row per parameter combination as a DataFrame
86 summary_table = (
87     full_results
88     .groupby(
89         ["heterogeneity_coef", "overlap_coef",
           "treatment_prevalence", "n_samples"]
90     )
91     .apply(summarise_results)
92     .reset_index()
93 )
94
95 # Save the summary table dataset
96 summary_table.to_csv(results_dir / "simulation_summary_results.csv",
           index=False)

```

A.2 module/data.py

Module for the data generating process.

```
1 from __future__ import annotations
2
3 import numpy as np
4 from scipy.special import expit
5
6
7 class SimulationData:
8     """
9     Generate simulation data with a binary, heterogeneous treatment
10     effect.
11
12     Uses the same data generating process as Yang et al., 2026.
13
14     The treatment effect is heterogeneous:
15      $\tau(x) = 1 + \text{heterogeneity\_coef} * x_1$ 
16
17     The treatment assignment follows a logistic propensity model:
18      $\text{logit}(e(x)) = \text{intercept} + \text{overlap\_coef} * (0.2 * X_1 + 0.3 * X_2$ 
19      $+ 0.4 * X_3 - 0.25 * X_4 - 0.3 * X_5 - 0.3 * X_6)$ 
20
21     And the outcome model given by:
22
23      $\mu(x) = (0.2 * X_1 + 0.3 * X_2 + 0.4 * X_3 - 0.25 * X_4 - 0.3 * X_5 -$ 
24      $0.3 * X_6) +$ 
25
26     Parameters
27     -----
28
29     n_samples:
30         Number of observations in sample
31     n_features:
32         Number of covariates/features.
33     rho:
34         Correlation coefficient between covariates.
35     overlap_coef:
36         Controls the amount of overlap in propensity scores.
37         Higher values create less overlap.
```



```

34 treatment_prevalence:
35     The target treatment rate. Intercept is shifted to achieve
        this.
36 heterogeneity_coef:
37     Coefficient on X_1 in tau(x). Higher values indicate stronger
38     heterogeneity.
39 noise_std:
40     Standard deviation of the random noise parameter. In all
        analyses, I
41     keep this equal to 1 to match Yang et al., 2026.
42
43 Returns
44 -----
45 dict
46     Contains:
47     - X: covariates
48     - T: binary treatment
49     - Y: observed outcome
50     - propensity: true propensity scores
51     - tau_true: true individual treatment effects
52     - ate_true: true sample ATE
53
54 """
55
56 def __init__(
57     self,
58     n_samples: int = 500,
59     n_features: int = 6,      # Keep at 6 to stay consistent with
        Yang et al., 2026
60     rho: float = 0.5,      # Keep at 0.5 to stay consistent with
        Yang et al., 2026
61     overlap_coef: float = 1.0, # use 1 and 3
62     treatment_prevalence: float = 0.4, # use 0.1 and 0.4
63     heterogeneity_coef: float = 0,
64     noise_std: float = 1.0, # keep at 1 to stay consistent with
        Yang et al., 2026
65 ) -> None:
66

```

```

67     self.n_samples = n_samples
68     self.n_features = n_features
69     self.rho = rho
70     self.overlap_coef = overlap_coef
71     self.treatment_prevalence = treatment_prevalence
72     self.heterogeneity_coef = heterogeneity_coef
73     self.noise_std = noise_std
74
75     def _make_covariance(self) -> np.ndarray:
76         """
77         Create covariance matrix. I simulate data such that all
78         covariates have
79         the same correlation with one another, for simplicity.
80         """
81         cov = np.full((self.n_features, self.n_features), self.rho,
82                       dtype=float)
83         np.fill_diagonal(cov, 1.0)
84         return cov
85
86     def _sample_features(self, rng: np.random.Generator) ->
87     np.ndarray:
88         """
89         Sample 6 latent variables
90
91         Sample X1-X3 continuous and X4-X6 binary from latent
92         Gaussian variables.
93         """
94         mean = np.zeros(self.n_features, dtype=float)
95         cov = self._make_covariance()
96
97         # Generate latent variables from a Gaussian distribution
98         V = rng.multivariate_normal(mean=mean, cov=cov,
99                                     size=self.n_samples)
100
101         # Keep the first three features as Gaussian
102         # Compute binary variables from the last three features
103         X = V.copy()
104         X[:, 3:6] = (V[:, 3:6] < 0).astype(float)

```

```

100
101     return X
102
103 def _propensity_linear(self, X: np.ndarray) -> np.ndarray:
104     """
105     Compute the linear predictor without the intercept. Weights
106     are chosen to
107     match Yang et al., 2026, to ensure meaningful comparisons.
108     i.e. overlap_coef * (0.2*X_1 + 0.3*X_2 + 0.4*X_3 - 0.25*X_4
109         - 0.3*X_5 - 0.3*X_6)
110
111     """
112     beta = np.array([0.2, 0.3, 0.4, -0.25, -0.3, -0.3],
113         dtype=float)
114     return self.overlap_coef * (X[:, : len(beta)] @ beta)
115
116 def _propensity_intercept(self, X: np.ndarray) -> float:
117     """
118     Shift the intercept of the treatment assignment model
119     to achieve the desired treatment prevalence.
120
121     Choose the intercept that achieves a mean propensity score
122     closest to target
123     """
124     linear_part = self._propensity_linear(X)
125     grid = np.linspace(-6.0, 6.0, 2000)
126     # compute the mean propensity score for each intercept in
127     # grid
128     mean_props = np.array([expit(a + linear_part).mean() for a
129         in grid])
130     # Choose 'a' to get closest to target rate.
131     idx = np.argmin(np.abs(mean_props -
132         self.treatment_prevalence))
133     return float(grid[idx])
134
135 def _true_tau(self, X: np.ndarray) -> np.ndarray:
136     """

```

```

131         Compute the true heterogeneous treatment effect tau(x).
132
133         This is used to compute bias and the CATE-RMSE for a trained
134             model.
135         """
136         return 1.0 + self.heterogeneity_coef * X[:, 0]
137
138     def _baseline_outcome(self, X: np.ndarray) -> np.ndarray:
139         """
140         Compute the baseline outcome function. Weights are chosen to
141             match
142             Yang et al. 2026, to ensure meaningful comparisons.
143
144         Used to compute the actual outcomes for storing.
145         """
146         beta_y = np.array([-0.5, -0.8, -1.2, 0.8, 0.8, 1.0],
147                             dtype=float)
148         return X[:, : len(beta_y)] @ beta_y
149
150     def generate(self, random_state: int | None = None) -> dict[str,
151         np.ndarray | float]:
152         """
153         Generate one simulated dataset.
154         """
155         rng = np.random.default_rng(random_state)
156
157         X = self._sample_features(rng)
158         intercept = self._propensity_intercept(X)
159
160         linear_part = self._propensity_linear(X)
161         propensity = expit(intercept + linear_part)
162         T = rng.binomial(1, propensity).astype(float)
163
164         tau_true = self._true_tau(X)
165         mu = self._baseline_outcome(X)
166         noise = rng.normal(0.0, self.noise_std, size=self.n_samples)
167
168         Y = mu + T * tau_true + noise

```

```
165     ate_true = float(np.mean(tau_true))
166
167     return {
168         "X": X,
169         "T": T,
170         "Y": Y,
171         "propensity": propensity,
172         "tau_true": tau_true,
173         "ate_true": ate_true,
174     }
```

A.3 module/metrics.py

Module with helper functions for computing performance metrics.

```
1 from __future__ import annotations
2
3 import numpy as np
4 import pandas as pd
5
6
7 def cate_rmse(tau_true: np.ndarray, tau_hat: np.ndarray) -> float:
8     """
9     Compute RMSE for individual treatment effect estimates.
10
11     Parameters
12     -----
13     tau_true:
14         True individual treatment effects. (CATE)
15
16     tau_hat:
17         Model estimated treatment effects. (CATE)
18     """
19     return float(np.sqrt(np.mean((tau_hat - tau_true) ** 2)))
20
21
22 def ate_bias(ate_true: float, ate_hat: float) -> float:
23     """
24     Compute bias of the overall estimated ATE.
25
26     Parameters
27     -----
28     ate_true:
29         True sample ATE.
30     ate_hat:
31         Model estimated ATE for sample.
32     """
33     return float(ate_hat - ate_true)
34
35
36 def summarise_results(results: pd.DataFrame) -> pd.Series:
```

```

37     """
38     Aggregate replication-level simulation results.
39
40     Expected columns:
41     - ate_true
42     - ate_hat
43     - ate_bias
44     - cate_rmse
45     """
46     return pd.Series(
47         {
48             "mean_ate_bias": results["ate_bias"].mean(),
49             "rmse_ate": float(np.sqrt(np.mean(results["ate_bias"] **
50             2))),
51             "mean_cate_rmse": results["cate_rmse"].mean(),
52             "sd_ate_hat": results["ate_hat"].std(ddof=1),
53             "mean_treatment_rate": results["treatment_rate"].mean(),
54         }
55     )

```

A.4 module/simulation.py

Module for running full simulation runs.

```
1 from __future__ import annotations
2
3 import numpy as np
4 import pandas as pd
5 from econml.dr import LinearDRLearner
6 from sklearn.linear_model import LinearRegression, LogisticRegression
7
8 from overlapsim.metrics import ate_bias, cate_rmse
9
10
11 def correct_drlearner(random_state: int | None = None) ->
    LinearDRLearner:
12     """
13     Build a correctly specified LinearDRLearner.
14     """
15     return LinearDRLearner(
16         model_propensity=LogisticRegression(
17             penalty=None,
18             solver="lbfgs",
19             max_iter=1000,
20             random_state=random_state,
21         ),
22         model_regression=LinearRegression(),
23         cv=3,
24         random_state=random_state,
25     )
26
27
28 def run_one(generator, random_state: int | None = None) -> dict:
29     """
30     Run one full replication:
31     1. simulate a dataset,
32     2. fit LinearDRLearner,
33     3. compute key performance metrics.
34
35
```



```

36 Parameters
37 -----
38
39 generator:
40     The simulated data generator class
41 random_state:
42
43
44 Returns:
45 -----
46 dict
47     Contains:
48     - ate_true: Ground truth ATE
49     - ate_hat: Model estimated ATE in sample
50     - ate_bias: Difference between actual and estimated ATE
51     - cate_rmse: RMSE for the estimated CATEs
52     - treatment_rate: The actual treated population in random
53       sample
54     - min_propensity: The minimum observed propensity score
55     - max_propensity: The maximum observed propensity score
56 """
57 data = generator.generate(random_state=random_state)
58
59 X = data["X"]
60 T = data["T"]
61 Y = data["Y"]
62 tau_true = data["tau_true"]
63 ate_true = data["ate_true"]
64
65 # Train the model
66 model = correct_drlearner(random_state=random_state)
67 model.fit(Y, T, X=X)
68
69 # Recover the estimated CATE for each observation in sample
70 tau_hat = np.asarray(model.effect(X), dtype=float).reshape(-1)
71 # Take the mean to recover the estimated ATE
72 ate_hat = float(np.mean(tau_hat))

```

```

73     return {
74         "ate_true": ate_true,
75         "ate_hat": ate_hat,
76         "ate_bias": ate_bias(ate_true, ate_hat),
77         "cate_rmse": cate_rmse(tau_true, tau_hat),
78         "treatment_rate": float(np.mean(T))
79     }
80
81
82 def run_simulation(generator, n_replications: int, base_seed: int =
123) -> pd.DataFrame:
83     """
84     Run many replications under a fixed DGP using a Monte Carlo
85     setup.
86
87     Parameters:
88     -----
89     generator:
90         The simulated data generator class.
91     n_replications:
92         Number of times to replicate the simulation on a new seed.
93     base_seed:
94         Seed number to use as first random state.
95     """
96     rows = []
97
98     for rep in range(n_replications):
99         # Call run_one and change the seed every replication
100         result = run_one(generator, random_state=base_seed + rep)
101
102         # Save metadata for clean bookkeeping
103         # Important for sanity checks once we save the results
104         result["replication"] = rep
105         result["n_samples"] = generator.n_samples
106         result["overlap_coef"] = generator.overlap_coef
107         result["treatment_prevalence"] =
            generator.treatment_prevalence
108         result["heterogeneity_coef"] = generator.heterogeneity_coef

```

```
108         rows.append(result)
109
110     # convert list of dictionaries to a dataframe and return
111     return pd.DataFrame(rows)
```